

From Reward Generation to Reward Self-Evolution: LLM-Bootstrapped Schema Search for Reinforcement Learning

Anonymous

Abstract

Large language models (LLMs) have recently been used to generate and refine reward functions for reinforcement learning. However, many existing approaches still treat reward design as reward code generation guided by scalar performance, human feedback, or trajectory preference signals. We argue that the next stage is reward self-evolution: an LLM-based reward design agent should bootstrap its own reward schema, diagnose policy failures from trajectories, remember which edits helped or hurt, and revise its reward design without using the environment’s official reward as iterative feedback. We propose EG-RSA, an oracle-reward-free reward self-evolution framework. In V1, we study a human-seeded constrained setting to verify the diagnosis-memory-edit-audit loop. In V2, we extend the framework to LLM-bootstrapped reward schema synthesis from primitive task interfaces, followed by validated schema evolution. EG-RSA combines semantic role attribution, outcome lessons, auditable schema edits, risk-aware validation, and rollback. Experiments on LunarLander are designed to analyze not only final policy performance, but also reward hacking, bad edits, recovery, and the safety–exploration trade-off in self-evolving reward search.

1 Introduction

Reinforcement learning (RL) relies on reward functions to translate task intent into an optimization objective. A well-designed reward should provide useful learning signals while remaining aligned with the intended behavior. In practice, reward design remains a central bottleneck: poorly specified rewards can lead to reward hacking, exploitation of dense shaping signals, or policies that maximize proxy feedback without solving the real task [Sutton and Barto, 2018, Amodei et al., 2016, Pan et al., 2022].

Recent work has shown that LLMs can assist reward design. Eureka uses LLMs to generate executable reward code and improve it through iterative in-context optimization [Ma et al., 2023]. Text2Reward generates dense reward functions from natural language task descriptions [Xie et al., 2023]. Auto MC-Reward uses trajectory analysis and reward critics to refine dense rewards in sparse-reward environments [Li et al., 2023]. CARD further introduces dynamic feedback and trajectory preference evaluation to improve LLM-driven reward design [Sun et al., 2024]. REvolve frames reward design as an evolutionary process guided by human feedback [Hazra et al., 2024]. These works move reward design beyond manual engineering and establish LLMs as powerful reward designers.

However, reward generation and reward refinement are not the end of the story. A reward design agent should not merely generate a new reward after each failure. It should diagnose why its current reward failed, identify which reward components shaped the policy, remember which previous edits helped or hurt, and decide whether to edit, continue training, repair, or roll back. More importantly, in realistic reward engineering, the official environment reward is often unavailable or should not be

used as the search feedback; otherwise the system risks optimizing reward design against the answer it is supposed to replace. This motivates an oracle-reward-free view of reward design.

We propose to formulate LLM-assisted reward design as reward self-evolution. In this paradigm, the LLM is not only a reward code generator, but an agent that maintains and revises a reward schema across training iterations. The agent observes trajectory-derived diagnostics, performs semantic role attribution, retrieves outcome lessons from memory, proposes schema-level edits, and validates them through risk-aware audit and rollback. Official environment reward, when available, is used only for post-hoc evaluation rather than iterative reward-search feedback.

This paper develops EG-RSA, an Experience-Guided Reward Self-Evolution Agent. We distinguish two settings. V1 is a human-seeded constrained setting: the initial reward schema and diagnostic interface are task-informed, and the system verifies the internal loop of diagnosis, memory, editing, audit, and recovery. V2 removes the strongest manual prior by adding an LLM bootstrap stage: given only a primitive task interface and a natural language task description, the LLM generates the initial reward schema and diagnostic predicates, which are then self-evolved through the EG-RSA loop.

The central design principle of V2 is not free-form reward rewriting, but validated schema synthesis. The LLM may propose new reward components and event predicates from primitive variables, but every formula and condition must pass static checks, signal-variation tests, scale validation, risk audit, and rollback. This increases reward-search freedom while preserving auditability and safety.

Our contributions are threefold:

1. **Oracle-reward-free reward self-evolution.** We formulate LLM-assisted reward design as a self-evolving agent loop that uses trajectory-derived diagnostics and edit-history memory rather than the environment’s official reward as iterative search feedback.
2. **LLM-bootstrapped reward schema synthesis.** We introduce a V2 bootstrap stage in which the LLM generates the initial reward schema and diagnostic predicates from a primitive task interface, reducing reliance on manually designed reward semantics.
3. **Constrained and risk-aware schema evolution.** We design a validated schema-evolution mechanism that combines semantic role attribution, outcome lessons, auditable schema edits, static validation, behavior risk audit, and rollback to balance search freedom with reward-hacking safety.

2 Related Work

2.1 Reward Design and Reward Shaping

Reward design is a long-standing challenge in reinforcement learning. Since the reward function defines the optimization objective, learned policies are highly sensitive to how task intent is translated into scalar feedback [Sutton and Barto, 2018]. Reward shaping studies how auxiliary signals can improve learning, with potential-based shaping providing conditions under which reward transformations preserve optimal policies [Ng et al., 1999]. Inverse reinforcement learning and apprenticeship learning infer rewards or policies from expert demonstrations [Ng and Russell, 2000, Abbeel and Ng, 2004]. These methods reduce some manual reward engineering, but typically require either hand-designed shaping structure or expert behavior.

EG-RSA addresses a different setting. It assumes that reward design itself is an iterative search process. Instead of inferring a reward from expert demonstrations or optimizing a fixed manually shaped reward, EG-RSA maintains and revises a reward schema using trajectory diagnostics, attribution, memory, and validation. V1 studies this loop under a human-seeded semantic interface, while V2 moves toward LLM-bootstrapped schema synthesis from primitive task descriptions.

2.2 LLM-Based Reward Generation and Reward Evolution

LLMs have recently been used to automate reward design. Eureka shows that LLMs can generate reward code and improve it through evolutionary optimization over reward programs [Ma et al., 2023]. Text2Reward generates dense reward functions from natural language task descriptions [Xie et al., 2023]. Auto MC-Reward combines reward design, reward criticism, and trajectory analysis to refine rewards from collected behavior [Li et al., 2023]. CARD uses dynamic feedback and trajectory preference evaluation to improve LLM-generated reward functions efficiently [Sun et al., 2024]. REvolve further studies reward evolution with LLMs guided by human feedback [Hazra et al., 2024].

These works demonstrate that LLMs can generate and refine rewards, but they differ from EG-RSA in the role assigned to the LLM reward designer. EG-RSA treats the LLM as a reward self-evolution agent. The agent does not only generate reward code; it maintains a schema, diagnoses failures, retrieves previous outcome lessons, proposes auditable schema edits, and validates them without using the official environment reward as iterative feedback. This shifts the emphasis from reward generation to reward self-evolution.

2.3 Self-Refinement and Memory-Augmented LLM Agents

EG-RSA is related to LLM self-improvement and memory-based agents. Self-Refine improves LLM outputs through iterative self-feedback and refinement without supervised training or reinforcement learning [Madaan et al., 2023]. Reflexion uses verbal feedback and episodic memory to improve language-agent behavior without updating model parameters [Shinn et al., 2023]. Voyager maintains an expanding skill library and uses iterative prompting to learn in Minecraft [Wang et al., 2023]. STOP studies a self-taught optimizer that improves its own scaffolding program through LLM-generated modifications [Zelikman et al., 2023]. Self-Rewarding Language Models explore the idea of models providing feedback to improve their own training process [Yuan et al., 2024].

These works motivate the broader idea of agentic self-improvement. EG-RSA applies this idea to reward design for reinforcement learning. Unlike self-refinement methods that revise text outputs or agents that store action skills, EG-RSA stores reward-edit outcome lessons: schema diffs, metric changes, failure modes, risk flags, and rollback decisions. These lessons are directly used to guide later reward-schema evolution.

2.4 Reward Hacking, Oracle Feedback, and Safe Reward Search

Reward hacking and reward misspecification occur when an agent optimizes a proxy reward while failing the intended objective [Amodei et al., 2016, Pan et al., 2022, Skalse et al., 2022]. This problem is especially relevant for LLM-generated rewards, because dense shaping terms may be easy to exploit and may not align with sparse task success. Recent work on reward-model ensembles, information-theoretic reward modeling, and Goodhart effects further illustrates that optimizing imperfect reward signals can remain unsafe even when the reward model is improved [Eisenstein et al., 2024, Langosco et al., 2024].

EG-RSA addresses this issue by separating iterative reward-search feedback from post-hoc oracle evaluation. The official environment reward is not used to guide schema edits; instead, the system uses trajectory-derived diagnostics, semantic role attribution, and hack-risk signals. This makes EG-RSA closer to realistic reward engineering, where the desired reward is precisely what must be designed. At the same time, EG-RSA does not remove safety constraints: V2 increases LLM freedom through schema synthesis, but every generated formula and event predicate must pass validation, risk audit, and rollback checks.

3 Method

3.1 Overview: From V1 to V2

EG-RSA formulates reward design as an iterative self-evolution process. Let S_k denote the reward schema at iteration k , and let R_{S_k} be the executable reward compiled from that schema. A policy is trained under R_{S_k} , trajectories are collected, policy behavior is diagnosed, and an LLM reward-design agent proposes a schema update. The goal is to search for reward schemas that induce policies aligned with the intended task, without using the official environment reward as iterative search feedback.

We distinguish two settings. In V1, the initial schema and diagnostic interface are human-seeded. This controlled setting isolates the internal mechanisms: semantic attribution, memory, constrained editing, risk audit, and rollback. In V2, the system first bootstraps the initial schema and diagnostic predicates using the LLM itself. The input to V2 is a primitive task interface rather than a hand-written reward schema.

3.2 Primitive Interface and LLM Bootstrap

V2 starts from a primitive interface $\mathcal{I}$ consisting of a natural language task description, primitive observation variables, primitive action variables, allowed formula functions, and forbidden signals. For LunarLander, primitive variables include position, velocity, angle, angular velocity, leg contacts, and engine actions. The interface intentionally avoids manually defined high-level predicates such as landing-region or stable-landing conditions.

The bootstrap agent generates an initial schema

$$S_0 = B_\theta(\mathcal{I}), \quad (1)$$

where B_θ is an LLM bootstrap agent. The generated schema contains reward components, event predicates, semantic roles, diagnostic predicates, and risk notes. This stage reduces reliance on manually specified reward semantics: if a landing-zone-like predicate is useful, it must be synthesized from primitive variables by the LLM.

3.3 Reward Schema Representation

A reward schema is represented as

$$S = (\mathcal{C}, \mathcal{E}, \mathcal{M}), \quad (2)$$

where $\mathcal{C}$ is a set of reward components, $\mathcal{E}$ is a set of event predicates or event rules, and $\mathcal{M}$ contains metadata such as semantic roles. In V2, components may include formula-based expressions generated by the LLM, while event predicates may be synthesized from primitive variables.

The compiled reward has the general form

$$R_S(s_t, a_t, s_{t+1}) = \sum_i w_i \phi_i(s_t, a_t, s_{t+1}) + \sum_j \beta_j \mathbf{1}[e_j(s_t, a_t, s_{t+1})], \quad (3)$$

where ϕ_i is a dense or conditional reward expression, e_j is an event predicate, and $\mathbf{1}[\cdot]$ is an indicator function.

3.4 Semantic Diagnosis and Outcome Memory

After training a policy under R_{S_k} , EG-RSA collects trajectories and computes semantic diagnostics. These include success evidence, terminal event evidence, contact instability, repeated-event risk, dense reward dominance, and other task-level behavioral signals. Reward role attribution maps component-level reward contributions to semantic roles such as dense guidance, stability quality, terminal success, safety constraint, and control cost.

Each evaluated edit produces an outcome lesson

$$\ell_k = (S_k, \Delta_k, S'_k, O_k, O'_k, F_k, d_k), \quad (4)$$

where Δ_k is the edit plan, O_k and O'_k are outcomes before and after the edit, F_k contains failure diagnostics, and d_k is the accept, reject, or rollback decision. Effective edits and regressive edits are both stored. Later iterations retrieve relevant lessons to avoid repeating harmful search directions and to reuse successful ones.

3.5 Validated Schema Evolution

At iteration k , the LLM proposes a schema update Δ_k . V1 mainly uses constrained operators such as weight changes, disabling components, and adding event rules. V2 expands this into validated schema synthesis. The LLM may propose formula components, conditional components, or event predicates, but only over allowed primitive variables and functions.

A candidate schema is obtained by

$$S'_k = \text{Apply}(S_k, \Delta_k). \quad (5)$$

Before training, the candidate edit is validated by multiple checks:

1. static expression validation over allowed variables and functions;
2. signal tests on collected or random trajectories to reject constant, zero, non-finite, or exploding rewards;
3. scale audit to prevent dense rewards from overwhelming terminal signals;
4. behavior risk audit to detect reward-hacking risks;
5. repair or rollback when validation fails.

This design increases freedom compared with fixed schema editing while avoiding unconstrained reward-code execution.

3.6 Algorithm Summary

Algorithm 1 summarizes EG-RSA-V2. The official environment reward is not used to select edits or guide reward self-evolution; it may be used only for post-hoc evaluation.

4 Experiments

4.1 Setup

We evaluate EG-RSA on LunarLander-v3, a continuous control task in the Gymnasium suite [Towers et al., 2024]. The agent must land a lunar module smoothly on a landing pad with low velocity, upright orientation, and stable ground contact (8-dim observation, 2-dim continuous action). All experiments use PPO [Schulman et al., 2017] with a fixed 1M-step training budget per iteration, running for 10 iterations with a DeepSeek-based LLM edit agent. All EG-RSA components are enabled: memory, attribution, hack detection, operator constraints. The oracle reward is used only for post-hoc evaluation. The default audit policy is strict.

Algorithm 1 EG-RSA-V2: LLM-Bootstrapped Reward Self-Evolution

Require: Primitive task interface $\mathcal{I}$, iterations K , memory store $\mathcal{L}$

```
1:  $S_0 \leftarrow \text{LLM-Bootstrap}(\mathcal{I})$ 
2: for  $k = 0$  to  $K - 1$  do
3:   Compile reward  $R_{S_k}$  from schema  $S_k$ 
4:   Train policy  $\pi_k$  under  $R_{S_k}$ 
5:   Collect trajectories and semantic outcome  $O_k$ 
6:   Diagnose failure modes  $F_k$  and role attribution  $A_k$ 
7:   Retrieve relevant lessons  $\mathcal{L}_k$ 
8:   Query LLM for schema update  $\Delta_k$ 
9:   Validate formulas, events, scale, and hacking risk
10:  if  $\Delta_k$  passes validation then
11:     $S'_k \leftarrow \text{Apply}(S_k, \Delta_k)$ 
12:    Train/evaluate candidate schema  $S'_k$ 
13:    if candidate accepted then
14:       $S_{k+1} \leftarrow S'_k$ 
15:      Store effective lesson in  $\mathcal{L}$ 
16:    else
17:       $S_{k+1} \leftarrow \text{rollback schema}$ 
18:      Store regression lesson in  $\mathcal{L}$ 
19:    end if
20:  else
21:    Repair edit or continue training with  $S_k$ 
22:     $S_{k+1} \leftarrow S_k$ 
23:  end if
24: end for
25: return best schema  $S^*$ 
```

4.2 Main Experiment and Case Studies

Table 1 presents the 10-iteration run under strict audit. The experiment exhibits three phases: an effective edit (Iter 0→1), a regression (Iter 1→2), and a prolonged low-success plateau (Iter 2→8). We examine each as a case study.

Effective edit (Iter 0→1). At iteration 0, the initial schema produces a policy achieving task score 0.478. Attribution identifies `r_approach_region` (dense guidance) as dominant (ratio 0.42). Two failure modes are detected: shaping-goal mismatch (no terminal success despite progress) and repeated event exploitation (rapid leg-contact toggling, 370 events/episode average). The LLM, informed by this diagnosis and having no prior lessons (first iteration), proposes: increase `r_landing_quality` weight, add a stable-landing event rule, decrease `r_approach_region`. The edit is classified medium-risk but permitted as a first-iteration exception. After training, task score reaches 0.867, `r_landing_quality` becomes dominant, and both failure modes are absent. The system records an effective-edit lesson.

Regression and audit deadlock (Iter 1→8). At iteration 2, the task score drops to 0.412, failure modes return, and `r_approach_region` again dominates. The system records a regression lesson (metric delta -0.455). Following this regression, the LLM repeatedly proposes edits that strengthen terminal success rewards and weaken dense guidance—the same strategy that succeeded at iteration 1. However, these edits are now classified as medium risk (modifying terminal reward

Table 1: EG-RSA 10-iteration run on LunarLander-v3 under strict audit.

Iter	Task	Semantic	Hack	Failures	Dominant Component
0	0.478	0.478	0.40	REE, SGM	r_approach_region
1	0.867	0.950	0.00	(none)	r_landing_quality
2	0.412	0.412	0.40	REE, SGM	r_approach_region
3	0.416	0.416	0.40	REE, SGM	r_landing_quality
4	0.367	0.367	0.40	REE, SGM	r_landing_quality
5	0.631	0.631	0.40	REE, SGM	r_approach_region
6	0.477	0.477	0.40	REE, SGM	r_landing_quality
7	0.422	0.422	0.20	SGM	r_landing_quality
8	0.665	0.665	0.40	REE, SGM	r_landing_quality

REE = repeated_event_exploitation; SGM = shaping_goal_mismatch.

Table 2: EG-RSA under relaxed audit.

Iter	Task	Semantic	Hack	Failures	Dominant Component
0	0.478	0.478	0.40	REE, SGM	r_approach_region
1	0.511	0.511	0.40	REE, SGM	r_approach_region
2	2.490	3.907	0.00	(none)	r_landing_quality
3	2.953	4.453	0.00	(none)	r_landing_quality

structure), and success evidence is weak (no terminal success since iteration 1). Under the strict audit policy, medium-risk edits with weak success evidence are blocked. The repair loop cannot resolve this: repairs adjust weights but cannot eliminate the risk classification. The result is a deadlock across iterations 2–8: scores remain between 0.367–0.665, failure modes persist, and the system cannot escape.

This deadlock is not an implementation flaw—it reveals a fundamental design tension. The same audit mechanism that prevents harmful edits can also block the exploration needed to establish success evidence. The system needs to modify terminal rewards to discover successful landing, but those modifications are precisely what strict audit blocks when success evidence is absent.

4.3 Relaxed Audit Policy

To test whether the deadlock is audit-induced, we run a second experiment with a relaxed policy: medium-risk edits under weak evidence are accepted with a warning. All other settings are identical.

Starting from the same initial schema (0.478), the relaxed-audit run follows a different trajectory. After a modest first edit (0.511), iteration 2 achieves a breakthrough: task score 2.490, no failure modes, **r_landing_quality** dominant. Iteration 3 continues to improve (2.953). These results confirm the deadlock was audit-induced: the same search mechanism—attribution, memory, constrained editing—produces sustained improvement once the audit barrier is lowered.

These results do not imply audit should be removed. Rather, they motivate a risk-budget mechanism: high-risk edits remain blocked, medium-risk edits are permitted with monitoring, and rollback serves as a safety net. Designing such a mechanism is future work.

5 Discussion

These experiments serve as mechanism verification, not a performance benchmark. Three findings emerge. First, attribution-guided editing with structured memory can produce effective reward modifications, as demonstrated by the iteration 0→1 case study. Second, regression is real in reward search and the system tracks it through structured outcome lessons. Third, the audit deadlock reveals a fundamental tension: integrating safety constraints into reward search creates a tradeoff between risk prevention and exploration, where strict blocking can suppress the edits needed to establish success evidence.

Our study has several limitations. We evaluate on a single environment (LunarLander-v3). Task metrics are template-instantiated rather than auto-discovered. Audit rules are hand-designed for this environment. Experiments run for 10 iterations; longer-horizon dynamics remain unexplored. We do not benchmark against EUREKA or other methods—such comparison requires multi-environment evaluation beyond this mechanism-verification scope.

The audit deadlock finding has implications beyond EG-RSA. Any LLM-based reward design system that introduces safety constraints—rule-based, learned, or ensemble-based—faces the same tradeoff. Strict blocking of medium-risk changes under weak evidence can suppress the edits needed to establish that evidence in the first place.

Future work includes multi-environment evaluation, learned audit rules that adapt to environment-specific failure modes, implementation of a risk-budget mechanism, and integration of process-level reward models [Zheng et al., 2025] to refine attribution granularity.

6 Conclusion

We reformulated LLM-assisted reward design as experience-guided schema search, introducing semantic role attribution, structured outcome memory, and integrated risk audit with rollback. Mechanism-verification experiments on LunarLander-v3 demonstrate effective editing, regression tracking, and an audit-induced deadlock that reveals a safety–exploration tradeoff. Resolving this deadlock through relaxed audit motivates future work on risk-budget mechanisms for safe reward search.

References

- Pieter Abbeel and Andrew Y. Ng. Apprenticeship learning via inverse reinforcement learning. In *Proceedings of the Twenty-first International Conference on Machine Learning*, 2004.
- Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in ai safety. *arXiv preprint arXiv:1606.06565*, 2016. URL <https://arxiv.org/abs/1606.06565>.
- Jacob Eisenstein, Chirag Nagpal, Alekh Agarwal, Ahmad Beirami, Alex D’Amour, Krishnamurthy Dvijotham, Adam Fisch, Katherine Heller, Stephen Pfohl, Deepak Ramachandran, et al. Helping or herding? reward model ensembles mitigate but do not eliminate reward hacking. In *Conference on Language Modeling (COLM)*, 2024. URL <https://arxiv.org/abs/2312.09244>.
- Rishi Hazra, Alkis Sygkounas, Andreas Persson, Amy Loutfi, and Pedro Zuidberg Dos Martires. Revolve: Reward evolution with large language models using human feedback. *arXiv preprint arXiv:2406.01309*, 2024. URL <https://arxiv.org/abs/2406.01309>.

- Lauro Langosco, Jan Wahlster, Felix Eberhardt, Nina Rimskey, and Rohin Shah. Catastrophic goodhart: Regularizing rlhf with kl divergence does not mitigate heavy-tailed reward misspecification. *arXiv preprint arXiv:2407.14503*, 2024. URL <https://arxiv.org/abs/2407.14503>. NeurIPS 2024.
- Hao Li, Xue Yang, Zhaokai Wang, Xizhou Zhu, Jie Zhou, Yu Qiao, Xiaogang Wang, Hongsheng Li, Lewei Lu, and Jifeng Dai. Auto mc-reward: Automated dense reward design with large language models for minecraft. *arXiv preprint arXiv:2312.09238*, 2023. URL <https://arxiv.org/abs/2312.09238>.
- Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models. *arXiv preprint arXiv:2310.12931*, 2023. URL <https://arxiv.org/abs/2310.12931>.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. *arXiv preprint arXiv:2303.17651*, 2023. URL <https://arxiv.org/abs/2303.17651>.
- Andrew Y. Ng and Stuart Russell. Algorithms for inverse reinforcement learning. In *Proceedings of the Seventeenth International Conference on Machine Learning*, pages 663–670, 2000.
- Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning*, pages 278–287, 1999.
- Alexander Pan, Kush Bhatia, and Jacob Steinhardt. The effects of reward misspecification: Mapping and mitigating misaligned models. *arXiv preprint arXiv:2201.03544*, 2022. URL <https://arxiv.org/abs/2201.03544>.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. URL <https://arxiv.org/abs/1707.06347>.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023. URL <https://arxiv.org/abs/2303.11366>.
- Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krashennnikov, and David Krueger. Defining and characterizing reward hacking. *Advances in Neural Information Processing Systems*, 35, 2022. URL <https://arxiv.org/abs/2209.13085>.
- Shengjie Sun, Runze Liu, Jiafei Lyu, Jing-Wen Yang, Liangpeng Zhang, and Xiu Li. A large language model-driven reward design framework via dynamic feedback for reinforcement learning. *arXiv preprint arXiv:2410.14660*, 2024. URL <https://arxiv.org/abs/2410.14660>.
- Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.

- Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024. URL <https://arxiv.org/abs/2407.17032>.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023. URL <https://arxiv.org/abs/2305.16291>.
- Tianbao Xie, Siheng Zhao, Chen Henry Wu, Yitao Liu, Qian Luo, Victor Zhong, Yanchao Yang, and Tao Yu. Text2reward: Reward shaping with language models for reinforcement learning. *arXiv preprint arXiv:2309.11489*, 2023. URL <https://arxiv.org/abs/2309.11489>.
- Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models. *arXiv preprint arXiv:2401.10020*, 2024. URL <https://arxiv.org/abs/2401.10020>.
- Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-taught optimizer (stop): Recursively self-improving code generation. *arXiv preprint arXiv:2310.02304*, 2023. URL <https://arxiv.org/abs/2310.02304>.
- Congming Zheng, Xiangyu Zhu, et al. A survey of process reward models: From outcome signals to process supervisions for large language models. *arXiv preprint arXiv:2510.08049*, 2025. URL <https://arxiv.org/abs/2510.08049>.

A Appendix: Extended Experiment Details

A.1 Environment and Hyperparameters

LunarLander-v3 provides an 8-dimensional observation space (position, velocity, angle, angular velocity, leg contact) and a 2-dimensional continuous action space (main engine, side engines). PPO hyperparameters: learning rate 3×10^{-4} , 64 parallel environments, 256-step rollout, 10 epochs per update, GAE $\lambda = 0.95$, clipping $\epsilon = 0.2$. Training budget: 10^6 environment steps per iteration.

A.2 Experiment Mode

All EG-RSA components are enabled: `use_memory=true`, `use_attribution=true`, `use_hack_detector=true`, `use_llm_edit=true`, `use_operator_constraints=true`, `free_rewrite=false`. The LLM edit agent is DeepSeek-based.

A.3 Extended Iteration Traces

Full iteration data including per-iteration edit plans, audit reports, repair logs, and outcome lesson cards are available in the experiment directories under `experiments/eg_rsa_lunar_lander_v1_role_attrib_10x1m` and `experiments/eg_rsa_lunar_lander_v1_role_attrib_10x1m_audit_relaxed/`.

A.4 Planned Ablations

Planned experiments include: disabling memory retrieval (feed-forward baseline), disabling attribution (scalar-feedback-only baseline), and removing operator constraints (free-code baseline). These ablations will isolate the contribution of each mechanism. Configurations exist in the `configs/` directory; experimental results are pending.